The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch02_python

AYuSh

Contents

Chapter 2: Python for ML Interviews

Python questions in an ML interview are rarely about Python for its own sake. They are a proxy for a sharper question: *can this person turn an idea into working, efficient, maintainable code?* A candidate who reaches for a generator when data doesn't fit in memory, who knows why `apply(axis=1)` is a thousand times slower than a vectorized expression, and whose data pipeline is testable rather than a 400-line script, is someone who can be trusted with production ML. That is what the interviewer is buying.

This chapter covers the Python that ML interviews actually test: the core language features that come up in screens (data structures, comprehensions, generators, decorators, OOP), the two libraries every ML role assumes (NumPy and Pandas) at the level where performance questions live, the complexity of common operations, and the habits that make ML code clean and testable. Algorithmic problem-solving — the LeetCode-style material — is Chapter 3; this chapter is about *fluency*.

Core Python

Built-in data structures: what they are and what they cost

Python's four workhorse containers each have a specific implementation, and interviewers expect you to know the cost model that follows from it.

A **list** is a dynamic array: a contiguous block of pointers to objects, over-allocated so that appending is cheap. Indexing and `append` are O(1) (append is *amortized* O(1) — occasionally the array is reallocated and copied), but inserting or deleting at the front is O(n) because every element shifts, and `x in my_list` is O(n) because there is nothing to do but scan. Lists are mutable and ordered; they are the default sequence.

A **tuple** is an immutable list. Immutability buys three things: tuples can be dictionary keys and set members (they are hashable, provided their contents are); they are slightly smaller and faster to create; and they communicate intent — a `(latitude, longitude)` pair is a fixed-shape record, not a growable collection. Returning multiple values from a function is tuple packing; `lat, lon = get_coords()` is unpacking.

A **dict** is a hash table: keys are hashed to slots, giving O(1) average insertion, lookup, and deletion. Since Python 3.7, dicts preserve insertion order as a language guarantee. The cost of hashing is worst-case degradation: adversarial or terrible hash functions make operations O(n), which is a classic follow-up question. In ML, dicts are everywhere the moment data stops being rectangular: vocabularies mapping token → id, configuration objects, feature registries, label maps.

A **set** is a dict with only keys: O(1) average membership testing, plus set algebra (union, intersection, difference). The two ML reflexes: deduplication (`set(user_ids)`) and fast membership (`if token in stopwords` — with a list of 500 stopwords this is the difference between O(1) and O(n) *per token*). Sets are unordered and their elements must be hashable.

Worked example. You process 1M tokens against 500 stopwords. With a stopword *list*: 1M × O(500) scans ≈ 5×10^8 comparisons. With a *set*: 1M × O(1) ≈ 10^6 hashes — a ~500× improvement from a one-character change. This is the most common "spot the inefficiency" bait in ML code screens.

Comprehensions: transformation as expression

A comprehension builds a container from an iterable in one readable expression:

```
squares  = [x**2 for x in range(10) if x % 2 == 0]   # list
vocab    = {tok: i for i, tok in enumerate(tokens)}  # dict
uniq_len = {len(w) for w in words}                   # set
lazy     = (x**2 for x in range(10**9))              # generator expression - no memory
```

Comprehensions are preferred over `map`/`filter` with lambdas in modern Python style, and are faster than an explicit `for` loop building a list with `.append` (the loop machinery is pushed into C). Two rules of taste that interviewers notice: don't nest more than two levels (extract a function instead), and don't use a comprehension *for its side effects* — a comprehension should build a value, not call `print` or mutate state.

The fourth form above — parentheses instead of brackets — is not a "tuple comprehension"; it is a **generator expression**, which brings us to the most important core-language topic for ML.

Generators and iterators: computing lazily

An **iterator** is any object that produces values one at a time via `__next__` and raises `StopIteration` when done; an **iterable** is anything that can produce an iterator via `__iter__`. The `for` loop is sugar over this protocol. A **generator** is the easy way to write an iterator: any function containing `yield` returns, when called, a paused computation that produces values on demand.

```
def read_records(path):
    with open(path) as f:
        for line in f:          # file objects are themselves lazy iterators
            yield parse(line)   # one record in memory at a time
```

Why ML interviews care: **training data usually does not fit in memory.** A generator-based pipeline — read a record, transform it, yield it, batch it — processes a 500 GB dataset with a constant memory footprint. This is precisely the design of PyTorch's `IterableDataset`, TensorFlow's `tf.data`, and every streaming feature pipeline. When an

interviewer asks "how would you train on data larger than RAM?", the first word of the answer is "stream": generators are the language-level mechanism.

The properties to state cold: generators are **lazy** (nothing computes until requested), **single-use** (once exhausted, they are empty — iterating twice is a real bug the code section demonstrates), and **cheap** (a generator object is ~100 bytes regardless of what it will produce). `yield` also supports pipelining: `sum(x**2 for x in stream)` composes without materializing anything. The `itertools` module (`islice`, `chain`, `cycle`, `groupby`) is the standard toolkit for combining them.

Decorators: functions that wrap functions

Python functions are first-class objects: they can be passed, returned, and stored. A **closure** is a function that captures variables from its enclosing scope. A **decorator** is a function that takes a function and returns a wrapped version — and `@decorator` is syntax sugar for `fn = decorator(fn)`.

```
def retry(times):
    def decorator(fn):
        @functools.wraps(fn)              # keep fn's name and docstring
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try:
                    return fn(*args, **kwargs)
                except TransientError:
                    continue
            raise
        return wrapper
    return decorator

@retry(times=3)
def call_feature_store(key): ...
```

The `*args, **kwargs` idiom makes the wrapper signature-agnostic: `*args` collects positional arguments into a tuple, `**kwargs` collects keyword arguments into a dict, and the wrapper forwards both untouched. `functools.wraps` copies the wrapped function's metadata onto the wrapper — omit it and every decorated function reports its name as `wrapper`, which breaks debugging and serialization.

Where decorators appear in ML systems: timing and logging (`@timed`), caching (`functools.lru_cache` — memoization in one line), retries around flaky I/O, input validation, registration (frameworks collect models or routes via `@register("resnet50")`), and PyTorch's `@torch.no_grad()` which switches off gradient tracking around evaluation code. Interviewers ask candidates to *write* a timing decorator often enough that the code section includes one.

OOP for ML: the sklearn pattern

ML interviews test OOP less as trivia ("what is polymorphism?") and more as design: *can you structure model code the way real libraries do?* The pattern to internalize is scikit-

learn's **estimator API**: a class with `fit(X, y)` that learns state (conventionally stored with a trailing underscore: `self.mean_`), `transform(X)` or `predict(X)` that applies it, and nothing else happening in `__init__` except storing hyperparameters. The power of the convention is composability — a `Pipeline` can chain any objects that follow it, and cross-validation can clone and refit them. The code section implements a `StandardScaler` this way, with tests.

Language mechanics worth having ready:

Dunder (double-underscore) methods define how objects respond to syntax. `__init__` constructs; `__repr__` displays; `__call__` makes an instance callable — the reason a PyTorch `model(x)` works; `__len__` and `__getitem__` make an object indexable and iterable — exactly the protocol of a PyTorch map-style `Dataset`, which is nothing more than a class defining those two methods.

Inheritance vs composition. Inherit when the subclass genuinely *is* a specialization sharing the parent's interface (`class ResNet(nn.Module)`); compose when a thing *has* a part (`Pipeline` has steps; a `Trainer` has a model, an optimizer, a scheduler). The interview-safe rule: prefer composition, use inheritance for framework base classes designed for it. Deep inheritance trees in ML code are a smell; two levels is usually plenty.

`@staticmethod` vs `@classmethod` vs instance methods. Instance methods receive `self`; classmethods receive `cls` and are the standard idiom for alternate constructors (`Model.from_pretrained(...)`, `Config.from_yaml(path)`); staticmethods receive neither and are namespaced utility functions.

Dataclasses. `@dataclass` autogenerates `__init__`, `__repr__`, and `__eq__` from field declarations — the modern way to write configuration objects (`TrainingConfig(lr=3e-4, batch_size=32, epochs=10)`) instead of dicts, gaining attribute access, defaults, and type hints that IDEs and reviewers can check.

NumPy

The ndarray: why it is fast

A Python list of a million floats is a million separate heap objects plus a million pointers; every arithmetic operation dispatches dynamically, object by object. A NumPy **ndarray** is one contiguous block of raw memory with a single **dtype** (say, `float64`), described by a **shape** and **strides** (how many bytes to step per dimension). Operations on it run as compiled C loops over machine numbers — no per-element interpretation, cache-friendly memory access, and SIMD where available. That design gap, not magic, is the 10–100× speedups this section measures.

Two consequences follow directly from the memory model and generate a disproportionate share of interview questions:

Views vs copies. Because an array is (shape, strides, pointer), many operations can return a new *view* of the same buffer instead of copying: basic slicing (`a[:, 1:3]`), `reshape` (usually), and `transpose` all do. Mutating a view mutates the original. **Fancy indexing** — indexing with an integer array or boolean mask — must gather scattered elements, so it always returns a *copy*. Knowing which is which prevents both accidental-mutation bugs and accidental-copy performance cliffs; the code section demonstrates both with `np.shares_memory`.

dtype discipline. `float64` is the default, but half the memory (`float32`) is the working standard in deep learning, and integer overflow is silent in fixed-width dtypes (`np.array([200], dtype=np.int8) + 100` wraps around to a negative number, with no exception). When a pipeline mysteriously doubles its memory, a stray `float64` upcast is the first suspect.

Vectorization: replace the loop with the array

Vectorization means expressing a computation as whole-array operations so the loop runs in C rather than in the interpreter. The interview rule: *if you wrote `for` over array elements, you probably made a mistake.*

The canonical transformation, measured in the code section: applying a sigmoid to a million values with a Python-level loop takes ~0.5 s; the expression `1.0 / (1.0 + np.exp(-z))` takes ~9 ms — a **54× speedup** for identical results. Everything in the expression — negation, exp, add, divide — is a **ufunc** (universal function) that maps over arrays elementwise in compiled code.

Vectorized thinking is a *translation skill*: conditional logic becomes `np.where(cond, a, b)`; accumulation becomes `np.cumsum`; "does any/all row satisfy X" becomes `mask.any(axis=1)`; counting becomes `(y_pred == y_true).mean()` (accuracy in one expression, exploiting the fact that booleans are 0/1). The reductions all take an **axis** argument, and axis semantics are a favorite quick check: for a matrix, `axis=0` collapses rows (result: one value per column), `axis=1` collapses columns (one value per row). "The axis you name is the axis that disappears" is the reliable mnemonic.

Broadcasting: arithmetic across mismatched shapes

Broadcasting is the rule NumPy uses to combine arrays of different shapes without copying data. Align the shapes *from the right*; two dimensions are compatible if they are equal, or if either is 1 (that dimension is conceptually stretched to match); missing leading dimensions are treated as 1.

```
X    (1000, 64)          X     (1000, 1, 64)
mu   (      64)  -> OK    C     (   1, 10, 64)  -> result (1000, 10, 64)
```

The left example is the everyday case: `X - X.mean(axis=0)` centers every column, the (64,) mean broadcasting across 1000 rows. The right example is the power move: insert size-1

axes with `None`/`np.newaxis` so that *all pairs* interact — `X[:, None, :] - C[None, :, :]` computes every point-minus-centroid difference in one expression, giving pairwise distances for k-means with no loop (code section, Listing 3).

Two hazards to volunteer in an interview. **Silent shape bugs:** subtracting a `(n,)` vector from a `(n, 1)` column doesn't error — it broadcasts to `(n, n)`, producing a plausibly-shaped wrong answer downstream. This is the argument for keeping vectors deliberately 1-D or deliberately 2-D and for `assert`-ing shapes at function boundaries. **Memory blowup:** broadcasting avoids copying *inputs*, but the *output* is fully materialized — the `(1000, 10, 64)` temporary above is fine, but the same pattern with 100k points and 10k centroids would try to allocate hundreds of GB. The fix is chunking, or algebra: expand the squared distance into norms and a matrix product (`||x-c||² = ||x||² + ||c||² - 2x·c`), which needs only a `(n, k)` result.

Array operations you must know

Beyond arithmetic, a short list of operations covers most interview code. `reshape(-1, d)` (the -1 means "infer this dimension"); `argmax`/`argmin` and `argsort` (indices, not values — `np.argsort(scores)[::-1][:k]` is top-k); boolean masking (`X[y == 1]` selects the positive class); `np.unique(labels, return_counts=True)` (class distribution in one call); `concatenate`/`stack` (join on an existing axis vs a new one); and `np.random.default_rng(seed)` — the modern, explicitly-seeded random API, which matters because reproducibility questions ("your results differ between runs — why?") are stock interview material. For linear algebra: `@` (matrix multiply), `np.linalg.solve` (never invert explicitly — Chapter 1), and `np.einsum` for anything with more than two indices (worth recognizing even if you don't write it fluently: `np.einsum('bik,bkj->bij', A, B)` is a batched matrix multiply, and attention implementations read this way).

Pandas

The DataFrame model

A **Series** is a 1-D array with an **index** (labels); a **DataFrame** is a dict of Series sharing one index — columns each have a dtype, rows are labeled. The index drives pandas' signature behavior, **alignment**: operations between Series match on labels, not positions, so adding two revenue Series from different months aligns by date and produces NaN where either is missing. Alignment is why pandas feels magical when indexes agree and baffling when they don't; `reset_index()` is the escape hatch back to plain positional data.

Two selection idioms must be automatic: `df.loc[rows, cols]` selects by *label* (inclusive slices), `df.iloc[rows, cols]` by *position* (exclusive slices, like Python). Mixing them up — or chaining selections like `df[df.x > 0]['y'] = 1`, which may modify a temporary copy instead of `df` and triggers the infamous `SettingWithCopyWarning` — is the most common pandas bug class; the fix is always a single `.loc`: `df.loc[df.x > 0, 'y'] = 1`.

groupby: split-apply-combine

`groupby` implements the **split-apply-combine** pattern: partition rows by key, apply a computation per group, stitch the results together. The interview-relevant distinctions are between the three apply-stage methods:

`agg` reduces each group to one row per statistic — `df.groupby('user_id').agg(revenue=('amount', 'sum'), n=('amount', 'size'))` produces per-user aggregates (and named aggregation, shown here, keeps columns tidy). **`transform`** returns a result *the same length as the input*, broadcast back to every row of the group — which is exactly what feature engineering needs: `df['amt_vs_user_mean'] = df.amount / df.groupby('user_id').amount.transform('mean')` attaches "this transaction relative to the user's average" to every transaction, no merge required. **`apply`** runs an arbitrary function per group — maximally flexible, minimally fast, the last resort.

`size` vs `count` is a small trap: `size` counts rows, `count` counts *non-null* values.

merge and join: SQL in memory

`pd.merge(left, right, on='key', how=...)` is a hash join. The four `how` modes match SQL: `inner` keeps matched keys only, `left` keeps all left rows (unmatched right columns become NaN), `right` mirrors it, `outer` keeps everything. The code section's pipeline shows the practical read: after a left join of per-user aggregates onto a segments table, a user present in events but absent from segments survives with `segment = NaN` — and a user with no events silently disappears, which is precisely the kind of row-count change to check after every merge (`len(result)` vs expectation, or `validate='one_to_one'` / `'many_to_one'` to make the assumption explicit and loudly enforced).

The failure mode interviewers probe: merging on a key that is non-unique on *both* sides produces the Cartesian product of matches — a 10-row × 10-row duplicate key becomes 100 rows, and at scale this is the "my join exploded to 40 GB" incident. State the guard: know which side is supposed to be unique, `drop_duplicates` first, and use `validate`.

pivot and reshape: wide vs long

`pivot_table(index=..., columns=..., values=..., aggfunc=...)` turns long data (one row per observation) into wide data (one row per entity, one column per category), aggregating duplicates — the code section builds a segment × event-type count table in one call. Its inverse is `melt`, wide back to long. The ML relevance: models want wide (one row per example), storage and event logs are long (one row per event), so real feature pipelines pivot constantly. `pd.crosstab(y_true, y_pred)` — a special-cased pivot — is a confusion matrix in one line.

Missing data

Pandas historically represents missingness with the float `NaN`, which imposes a gotcha: an integer column acquiring a NaN silently becomes float (nullable dtypes like `Int64` fix this, at the cost of remembering they exist). The mechanics: `isna()` to detect (never `== np.nan`, which is always False — NaN compares unequal to everything including itself), `dropna()` to remove, `fillna(value)` to impute, and `NaN` is contagious through arithmetic while being *skipped by default* in reductions (`mean(skipna=True)`) and *dropped by default* by groupby keys — both defaults can quietly bias an analysis.

The ML-critical points: first, **imputation is a model decision, not a cleanup chore** — filling with the median changes the distribution, filling with 0 invents a signal (the code section fills view-events' amounts with 0 because "views have no purchase amount" is *structural* missingness, a semantically different case from "amount lost in logging"); often the strongest move is adding a `was_missing` indicator column and letting the model use it. Second, **impute with statistics computed on the training set only** — computing the median over all data before splitting is leakage (Chapter 9 treats this fully).

apply vs vectorized operations

`df.apply(fn, axis=1)` calls a Python function once per row, constructing a Series for each — it is a Python loop wearing a pandas costume. The code section measures the same multiplication done three ways; the vectorized column expression is **~2,000× faster** than row-wise apply on a million rows. The escalation ladder when you're tempted to `apply`: (1) a vectorized column expression (`df.price * df.qty`); (2) `np.where` / `np.select` for conditionals; (3) `.map(dict)` for lookups; (4) `.str` / `.dt` accessor methods for text and dates; (5) only then `apply`, and prefer column-wise (`axis=0`) over row-wise. If genuinely stuck with per-row logic at scale, drop to NumPy arrays or vectorize the function itself.

Time and space complexity of common operations

Interviewers rarely ask for formal proofs here; they ask "what's the cost of X?" mid-discussion and listen for instant, correct answers. The table below is the required inventory. (Big-O review and algorithm design live in Chapter 3.)

Operation	Average complexity	Notes
`lst[i]`, `lst.append(x)`	O(1)	append amortized (occasional realloc + copy)
`lst.insert(0, x)`, `lst.pop(0)`	O(n)	shifts everything; use `collections.deque` for O(1) ends
`x in lst`	O(n)	linear scan
`x in set_` / `d[key]`	O(1)	hash table; O(n) worst case (collisions)

Operation	Average complexity	Notes
`sorted(lst)`, `lst.sort()`	O(n log n)	Timsort; O(n) on nearly-sorted data; stable
`heapq` push/pop	O(log n)	top-k pattern: heap of size k → O(n log k)
string concat in a loop	$O(n^2)$ total	strings immutable → each `+=` copies; use `''.join(parts)`
slice `lst[a:b]`	O(b−a)	copies (list); NumPy slicing is O(1) — returns a view
`np.dot(A, B)`, `A @ B`	O(m·n·p)	(m×n)(n×p); BLAS constant factors are tiny
elementwise ufunc on n elements	O(n)	C-speed; the constant is ~100× smaller than a Python loop
`np.argsort`	O(n log n)	`np.argpartition` gives unordered top-k in O(n)
`pd.merge` (hash join)	~O(n+m)	plus output size — which is O(n·m) if keys duplicate on both sides!
`groupby().agg()`	~O(n)	hash-based grouping

Space costs worth stating: a Python float is ~24 bytes of object overhead plus pointer versus 8 bytes in a `float64` array (and 4 in `float32`) — the reason "load it into NumPy" is itself a memory optimization; list over-allocation runs ~12%; a dict costs roughly 2–4× the raw key/value memory in table overhead; and every pandas operation that returns a new DataFrame (most do) briefly holds two copies, which is why memory-bound pipelines chunk their work or move to generators.

The top-k idiom deserves its sixty seconds: sorting all n scores to take k is O(n log n); a size-k heap is O(n log k); `np.argpartition(scores, -k)` is O(n). Recommending the right one for "find the 10 nearest neighbors among 10M vectors" is a stock systems-flavored coding question.

Writing clean, testable ML code

ML code rots in specific, predictable ways — hidden global state, irreproducible randomness, leaky preprocessing, and notebooks that became load-bearing. Interviewers increasingly test for the antibodies directly ("how would you test this pipeline?"), and strong answers are concrete:

Structure: config → data → model → train → evaluate. Separate these stages into functions or small classes with explicit inputs and outputs; keep hyperparameters in one dataclass rather than scattered literals; make every function referentially honest (same inputs → same outputs) except at clearly marked randomness boundaries. The sklearn estimator convention (hyperparameters in `__init__`, learned state as `attr_` set by `fit`, no I/O inside the class) is the pattern to name.

Reproducibility. Seed everything explicitly (`np.random.default_rng(seed)` passed in, not global `np.random.seed` sprinkled around), pin dependency versions, log the config with every result. "Your metric changed and no code changed — what happened?" is a stock question; unseeded shuffling, dict-ordering assumptions in old versions, and nondeterministic GPU kernels are the expected suspects.

Tests for ML code differ from generic unit tests, and naming the categories is a strong signal. *Shape and dtype tests*: the transform outputs (n, d) float32, no NaNs. *Statistical property tests*: standardized output has mean ≈ 0, std ≈ 1 (Listing 7 does exactly this). *Contract tests*: transform before fit raises; constant column doesn't crash. *Leakage tests*: transforming a held-out point uses training statistics (Listing 7's `test_no_leakage_train_stats_reused` — arguably the highest-value five-line test in ML). *Overfit-one-batch test*: a working model driven to near-zero loss on 10 examples; if it can't, the wiring is broken — the fastest smoke test in deep learning. *Golden/regression tests*: pin a small end-to-end run's metric within a tolerance.

Fail loudly at boundaries. `assert X.ndim == 2`, validate merges, raise on unexpected NaN influx. Silent shape coercion and silent NaN propagation are how ML bugs survive to production; a one-line assert converts a week of debugging into a stack trace.

Code implementations

Every listing below was executed as printed; outputs are real.

Listing 1 — Generators: constant memory and single-use semantics

```python
import sys

# Eager: materialize a million squared numbers
eager = [i * i for i in range(1_000_000)]
# Lazy: a recipe that produces them one at a time
lazy = (i * i for i in range(1_000_000))

print(f"list size: {sys.getsizeof(eager)/1e6:.1f} MB")
print(f"generator size: {sys.getsizeof(lazy)} bytes")

def batch_iter(data, batch_size):
    """Yield successive batches without copying the dataset."""
    for start in range(0, len(data), batch_size):
        yield data[start:start + batch_size]   # yield pauses here, resumes on next()

batches = batch_iter(list(range(10)), batch_size=4)
print("batches:", [b for b in batches])
print("exhausted:", [b for b in batches])   # generators are single-use!
```

Output:

```
list size: 8.4 MB
generator size: 104 bytes
batches: [[0, 1, 2, 3], [4, 5, 6, 7], [8, 9]]
exhausted: []
```

An 8.4 MB list versus a 104-byte generator *for the same computation* — that gap is why data loaders stream. `batch_iter` is a five-line version of what every ML framework's loader does (note it yields a final short batch — decide deliberately whether to keep or drop it). The last line is the classic bug: the second iteration silently produces nothing, which in real code looks like "my second epoch trained on zero batches."

Listing 2 — Decorators: timing and memoization

```python
import functools, time

def timed(fn):
    """Decorator: log how long fn takes, without touching its code."""
    @functools.wraps(fn)                        # preserve fn.__name__, docstring
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        result = fn(*args, **kwargs)
        print(f"{fn.__name__}: {time.perf_counter() - t0:.4f}s")
        return result
    return wrapper

def memoize(fn):
    """Cache results by argument - dynamic programming for free."""
    cache = {}
    @functools.wraps(fn)
    def wrapper(*args):
        if args not in cache:
            cache[args] = fn(*args)
        return cache[args]
    return wrapper

@memoize
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

@timed
def run():
    return fib(300)

print("fib(300) has", len(str(run())), "digits")
```

Output:

```
run: 0.0010s
fib(300) has 63 digits
```

Without memoization, naive `fib(300)` would need $\sim 2^{300}$ calls — the memoized version finishes in a millisecond because each subproblem is computed once and cached in the closure's dict. `memoize` is a hand-rolled `functools.lru_cache` (use the built-in in real code; write it by hand in interviews). Note the two-layer structure of `timed`: the outer function receives `fn`, the inner `wrapper` receives the call arguments, and the closure connects them.

Listing 3 — Vectorization and broadcasting

```
import numpy as np, time
rng = np.random.default_rng(42)
z = rng.normal(size=1_000_000)

t0 = time.perf_counter()
out_loop = [1.0 / (1.0 + np.exp(-v)) for v in z]      # Python-level loop
t_loop = time.perf_counter() - t0

t0 = time.perf_counter()
out_vec = 1.0 / (1.0 + np.exp(-z))                    # one C-level pass
t_vec = time.perf_counter() - t0

print(f"loop: {t_loop:.2f}s | vectorized: {t_vec*1000:.0f} ms | "
      f"speedup: {t_loop/t_vec:.0f}x")
print("identical:", np.allclose(out_loop, out_vec))

# Broadcasting: pairwise distances between 1000 points and 10 centroids
X = rng.normal(size=(1000, 64)); C = rng.normal(size=(10, 64))
# (1000, 1, 64) - (1, 10, 64) -> broadcasts to (1000, 10, 64)
D = np.sqrt(((X[:, None, :] - C[None, :, :]) ** 2).sum(axis=2))
print("distance matrix shape:", D.shape,
      "| nearest centroid of first 3 points:", D[:3].argmin(axis=1))
```

Output:

```
loop: 0.47s | vectorized: 9 ms | speedup: 54x
identical: True
distance matrix shape: (1000, 10) | nearest centroid of first 3 points: [4 5 7]
```

The sigmoid comparison is the whole vectorization argument in six lines: same math, same results, 54× apart. The distance computation is the assignment step of k-means (Chapter 8) with zero loops: two `None`-inserted axes make the shapes broadcast-compatible, the subtraction materializes all 1000×10 difference vectors, and the axis-2 reduction collapses them to distances.

Listing 4 — Views vs copies

```
import numpy as np
a = np.arange(12).reshape(3, 4)
s = a[:, 1:3]          # slicing returns a VIEW - shares memory
s[0, 0] = 99           # mutates a as well
print("a[0] after editing the slice:", a[0])
f = a[a > 5]           # boolean (fancy) indexing returns a COPY
f[0] = -1
print("a unchanged by editing the copy:", a[1])
print("view shares memory:", np.shares_memory(a, s),
      "| fancy copy shares:", np.shares_memory(a, f))
```

Output:

```
a[0] after editing the slice: [ 0 99  2  3]
a unchanged by editing the copy: [4 5 6 7]
view shares memory: True | fancy copy shares: False
```

Writing 99 into the slice changed the original array — that's a view doing exactly what views do. When you need an independent slice, say `.copy()` explicitly. `np.shares_memory` is the diagnostic to name when asked "how would you check?"

Listing 5 — A pandas pipeline: missing data, groupby, merge, pivot

```
import numpy as np
import pandas as pd

events = pd.DataFrame({
    "user_id": [1, 1, 2, 2, 2, 3, 4, 4],
    "event":   ["view", "buy", "view", "view", "buy", "view", "buy", "buy"],
    "amount":  [np.nan, 20.0, np.nan, np.nan, 35.0, np.nan, 15.0, np.nan],
})
users = pd.DataFrame({
    "user_id": [1, 2, 3, 5],
    "segment": ["free", "pro", "free", "pro"],
})

# 1. Missing data: views have no amount - structural, so fill with 0
print("missing amounts:", events["amount"].isna().sum())
events["amount"] = events["amount"].fillna(0.0)

# 2. groupby: split-apply-combine
per_user = (events.groupby("user_id")
                  .agg(n_events=("event", "size"),
                       n_buys=("event", lambda s: (s == "buy").sum()),
                       revenue=("amount", "sum"))
                  .reset_index())
print(per_user)

# 3. merge: LEFT join keeps user 4 (no segment); user 5 (no events) drops
full = per_user.merge(users, on="user_id", how="left")
print("segment NaN for unmatched user:", full.loc[full.user_id == 4,
"segment"].isna().item())

# 4. pivot table: segment x event counts
pivot = events.merge(users, on="user_id", how="inner").pivot_table(
    index="segment", columns="event", values="user_id", aggfunc="count", fill_value=0)
print(pivot)
```

Output:

```
missing amounts: 5
   user_id  n_events  n_buys  revenue
0        1         2       1     20.0
1        2         3       1     35.0
2        3         1       0      0.0
3        4         2       2     15.0
segment NaN for unmatched user: True
event    buy  view
segment
free       1     2
pro        1     2
```

Eight rows of events and four users exercise the full toolkit. Follow one user through: user 4 has two buys totaling 15.0 (one buy amount was missing → filled to 0 — a judgment call the comment documents), survives the left join with a NaN segment, and is *absent* from the pivot because the pivot's inner merge drops users without segments.

Every row-count change here is intentional and checked; in production the same three operations run on millions of rows, and the discipline is identical.

Listing 6 — apply vs vectorized: the 2,000× gap

```
import numpy as np, pandas as pd, time
rng = np.random.default_rng(0)
df = pd.DataFrame({"price": rng.uniform(1, 100, 1_000_000),
                   "qty": rng.integers(1, 10, 1_000_000)})

t0 = time.perf_counter()
r1 = df.apply(lambda row: row["price"] * row["qty"], axis=1)   # row-wise Python loop
t_apply = time.perf_counter() - t0

t0 = time.perf_counter()
r2 = df["price"] * df["qty"]                                   # vectorized
t_vec = time.perf_counter() - t0

print(f"apply(axis=1): {t_apply:.2f}s | vectorized: {t_vec*1000:.1f} ms | "
      f"speedup: {t_apply/t_vec:.0f}x")
print("identical:", np.allclose(r1, r2))
```

Output:

```
apply(axis=1): 5.64s | vectorized: 2.6 ms | speedup: 2188x
identical: True
```

Not a typo: **two thousand times slower** for the same multiplication. `apply(axis=1)` constructs a Series object per row and crosses the Python/C boundary a million times; the column expression crosses it once. If a candidate remembers one pandas fact from this chapter, this is the one.

Listing 7 — A testable transformer, sklearn-style

```
import numpy as np

class StandardScaler:
    """Minimal sklearn-style transformer: fit on train, transform anywhere."""

    def fit(self, X):
        X = np.asarray(X, dtype=float)
        self.mean_ = X.mean(axis=0)
        self.scale_ = X.std(axis=0)
        self.scale_[self.scale_ == 0.0] = 1.0   # constant column: avoid 0-division
        return self                              # enable scaler.fit(X).transform(X)

    def transform(self, X):
        if not hasattr(self, "mean_"):
            raise RuntimeError("call fit() before transform()")
        return (np.asarray(X, dtype=float) - self.mean_) / self.scale_

    def fit_transform(self, X):
        return self.fit(X).transform(X)


# ---- tests: small, deterministic, one behavior each ----
def test_output_is_standardized():
    rng = np.random.default_rng(0)
```

```
    Z = StandardScaler().fit_transform(rng.normal(5, 3, size=(200, 4)))
    assert np.allclose(Z.mean(axis=0), 0, atol=1e-9)
    assert np.allclose(Z.std(axis=0), 1, atol=1e-9)

def test_no_leakage_train_stats_reused():
    scaler = StandardScaler().fit([[0.0], [10.0]])       # mean 5, std 5
    assert np.allclose(scaler.transform([[15.0]]), [[2.0]])  # uses TRAIN stats

def test_constant_column_does_not_crash():
    Z = StandardScaler().fit_transform([[3.0], [3.0], [3.0]])
    assert np.allclose(Z, 0.0)

def test_transform_before_fit_raises():
    try:
        StandardScaler().transform([[1.0]])
        assert False, "should have raised"
    except RuntimeError:
        pass

for t in [test_output_is_standardized, test_no_leakage_train_stats_reused,
          test_constant_column_does_not_crash, test_transform_before_fit_raises]:
    t(); print(f"PASS {t.__name__}")
```

Output:

```
PASS test_output_is_standardized
PASS test_no_leakage_train_stats_reused
PASS test_constant_column_does_not_crash
PASS test_transform_before_fit_raises
```

Twenty lines of class, four tests, and each test encodes a real failure mode: statistical correctness, leakage (the transform of a new point must use *training* statistics — the test would catch a refit-on-transform bug immediately), the divide-by-zero edge case, and the usage contract. In a real project these run under `pytest`; the pattern — tiny deterministic inputs, one behavior per test, assertions on properties rather than exact matrices — is what "how would you test ML code?" wants to hear.

Listing 8 — The mutable default argument

```
def add_feature(row, features=[]):     # BUG: default evaluated once, shared forever
    features.append(row)
    return features

print(add_feature("age"))
print(add_feature("income"))           # surprise: 'age' is still there

def add_feature_fixed(row, features=None):
    features = [] if features is None else features
    features.append(row)
    return features

print(add_feature_fixed("age"))
print(add_feature_fixed("income"))
```

Output:

```
['age']
['age', 'income']
```

```
['age']
['income']
```

Default argument values are evaluated **once, at function definition** — every call without the argument shares the same list, which accumulates state across calls. This is the single most-asked Python gotcha in interviews, and it bites real ML code (feature lists, config dicts as defaults). The `None` sentinel is the standard fix.

Pitfalls, comparisons and practical tips

Confusion	Resolution
list vs tuple	Mutable vs immutable. Tuples are hashable (usable as dict keys/set members), slightly cheaper, and signal fixed structure.
`is` vs `==`	`is` compares identity (same object), `==` compares value. Only use `is` for `None`. Small-int caching makes `a is b` "work" for 5 and fail for 500 — never rely on it.
shallow vs deep copy	`list(x)`, `x.copy()`, `copy.copy` duplicate the container but share nested objects; `copy.deepcopy` recurses. Nested-config mutation bugs live here.
view vs copy (NumPy)	Slices/reshape/transpose → views (mutations propagate); fancy/boolean indexing → copies. Check with `np.shares_memory`; force with `.copy()`.
`(n,)` vs `(n,1)`	1-D vs 2-D column. Mixing them broadcasts to `(n,n)` silently. Assert shapes at function boundaries.
`axis=0` vs `axis=1`	The axis you name is the axis that disappears: `axis=0` collapses rows → per-column result; `axis=1` collapses columns → per-row result.
`loc` vs `iloc`	Label-based (inclusive slices) vs position-based (exclusive). Chained indexing + assignment → `SettingWithCopyWarning`; always assign through a single `.loc`.
`agg` vs `transform` vs `apply` (groupby)	One row per group vs same-length broadcast (feature engineering) vs arbitrary-and-slow.
`size` vs `count`	Rows vs non-null values.
`NaN == NaN`	Always False. Use `isna()`. NaN also coerces int columns to float.
merge row-count changes	Left join preserves left row count only if the right key is unique — duplicate keys multiply rows. Use `validate=`, check `len()` after.
mutable default argument	Evaluated once at def-time and shared across calls. Sentinel: `def f(x, acc=None)`.
generator reuse	Generators are single-use; a second loop over one yields nothing. Re-create it or materialize a list deliberately.
string `+=` in a loop	$O(n^2)$. Collect in a list, `''.join(parts)`.

Confusion	Resolution
the GIL	One thread executes Python bytecode at a time → threads don't speed up pure-Python CPU work. NumPy/pandas release the GIL in C sections; use `multiprocessing` (or vectorize) for CPU-bound Python.

Practical tips for the live-coding portion: narrate your data-structure choices as you make them ("set, because I need membership tests") — the choice *is* the answer being graded. When writing NumPy in front of an interviewer, say shapes out loud at every step; shape errors caught verbally are free, shape errors caught by traceback cost minutes. And when asked to speed something up, profile before optimizing — `%timeit` on the suspect line, not guesswork — then reach for vectorization first, algorithmic improvements second, parallelism last.

Interview questions and answers

Core Python

Q1. When would you choose a tuple over a list, and why can tuples be dict keys when lists can't?

Choose a tuple for fixed-shape records (coordinates, RGB values, function multi-returns), when you need hashability, or to signal immutability. Dict keys must be hashable — the hash must stay constant for the table to find the key again. A list can mutate after insertion, changing what its hash *would* be, so lists are unhashable by design; a tuple's contents can't be reassigned (note: a tuple containing a list is still unhashable — hashability is recursive).

Q2. How does a Python dict achieve O(1) lookup, and when does that guarantee fail?

Keys are hashed to an index in a sparse table; lookup hashes the query, jumps to the slot, and resolves collisions by open addressing (probing). Average O(1) assumes hashes spread keys evenly. It degrades toward O(n) when many keys collide — pathological or adversarial hash inputs (the reason Python randomizes string hashing per process) — and every dict operation pays a rehash/resize occasionally as the table grows. *Interviewers listen for: hash → slot → collision handling, plus one failure mode.*

Q3. What's the difference between an iterable, an iterator, and a generator?

An iterable can produce an iterator (`__iter__`) — lists, dicts, files. An iterator produces values via `__next__` and raises StopIteration when exhausted; it's the thing a for loop actually drives. A generator is an iterator created by a function containing `yield` (or a genexp): the function body runs lazily, pausing at each yield. All generators are iterators; all iterators are iterable; a list is iterable but not an iterator (you can loop over it repeatedly).

Q4. Write (or describe) a decorator that caches a function's results. What are the limitations?

Closure over a dict keyed by args: if `args not in cache`, compute and store; return `cache[args]` (Listing 2). Limitations: arguments must be hashable (no lists/dicts/arrays — a NumPy array argument breaks it); the cache grows without bound (`functools.lru_cache(maxsize=...)` fixes this with LRU eviction); keyword-argument order and default-value handling need care; and caching an impure function (I/O, randomness) silently changes semantics.

Q5. Explain `*args` and `**kwargs`. Why do decorators almost always use them?

In a signature, `*args` packs surplus positional arguments into a tuple and `**kwargs` packs surplus keyword arguments into a dict; at a call site the same syntax unpacks. Decorators wrap arbitrary functions, so the wrapper can't know the wrapped signature — `def wrapper(*args, **kwargs): return fn(*args, **kwargs)` forwards any call unchanged. Same idiom for subclass methods that extend a parent's (`super().__init__(**kwargs)`).

Q6. What is the GIL, and what does it mean for speeding up ML preprocessing?

The Global Interpreter Lock allows only one thread to execute Python bytecode at a time, so threading doesn't parallelize CPU-bound Python code (it does help I/O-bound work — threads waiting on network/disk release the GIL). For CPU-bound preprocessing: vectorize into NumPy/pandas (their C loops release the GIL and are the biggest win anyway), or use `multiprocessing`/`joblib` (separate processes, separate GILs — with data-serialization overhead between them). PyTorch DataLoader workers are processes for exactly this reason.

Q7. What does `@property` do, and where is the pattern useful in ML code?

It exposes a method as an attribute: `model.n_params` computes on access, no parentheses. Useful for derived quantities that should read like state but must stay consistent with underlying data — `scaler.n_features_` derived from `mean_.shape`, a config's computed `steps_per_epoch`. It also lets you add validation to attribute assignment later (via the setter) without changing the class's public interface.

Q8. Why does PyTorch's Dataset require `__len__` and `__getitem__`, conceptually?

Those two dunders make an object obey the sequence protocol: `len(ds)` and `ds[i]` work, which is all a DataLoader needs to sample indices, fetch examples (possibly in worker processes), and batch them. It's duck typing as API design: any object implementing the protocol — wrapping files, databases, arrays — plugs in with no inheritance required (subclassing `Dataset` is convention and type-checking sugar). `__call__` plays the same role for models and transforms: `model(x)` works because `nn.Module` defines `__call__` (which adds hooks around your `forward`).

NumPy

Q9. Why is NumPy so much faster than pure Python for numeric work? Give the memory-level answer.

A list stores pointers to boxed heap objects — each float access follows a pointer, dispatches type checks, and thrashes cache. An ndarray stores raw fixed-dtype values contiguously; a ufunc runs one compiled C loop over the buffer with sequential (prefetch-friendly, SIMD-able) access and zero per-element interpretation. Same asymptotic complexity, ~100× constant factor. *Interviewers listen for: boxing/pointers vs contiguous buffer, interpreter dispatch vs C loop — not just "it's in C."*

Q10. State the broadcasting rules, and give the output shape of adding (3, 1, 5) and (4, 5).

Align shapes from the trailing dimension; dimensions are compatible if equal or if either is 1 (stretched conceptually, no copy); missing leading dims count as 1. Here: (3, 1, 5) vs (1, 4, 5) → last dim 5=5 ✓; middle 1 vs 4 → stretch to 4; leading 3 vs 1 → stretch to 3. Result: **(3, 4, 5)**. Incompatible example: (3, 2) + (3,) fails — trailing 2 vs 3.

Q11. You slice an array, modify the slice, and the original changes. Explain, and say how you'd prevent it.

Basic slicing returns a view — new shape/strides metadata over the same buffer — so writes are visible through both names (Listing 4). Prevent with an explicit `.copy()`. The asymmetric partner fact: fancy and boolean indexing return copies, so the reverse bug exists too — writing into `a[mask][0]` modifies a temporary and is lost (use `a[mask] = ...` directly, a single `__setitem__`, instead).

Q12. Compute row-wise softmax of a (n, k) score matrix in NumPy, numerically safely.

`z = S - S.max(axis=1, keepdims=True); e = np.exp(z); p = e / e.sum(axis=1, keepdims=True)`. The max subtraction prevents `exp` overflow (Chapter 1's log-sum-exp trick — softmax is shift-invariant so results are identical); `keepdims=True` keeps the reductions (n, 1) so they broadcast back across columns instead of misaligning. Both details are exactly what's being tested.

Q13. How would you find the 10 largest values in an array of 10 million, and what's the complexity?

`np.argpartition(x, -10)[-10:]` — O(n) selection that guarantees the top 10 are in the last 10 positions, unordered; sort those 10 afterwards if order matters (O(n + k log k)). Full `np.argsort` is O(n log n) — needless work for k ≪ n. Pure-Python equivalent: `heapq.nlargest(10, x)`, O(n log k). Recommending partition-over-sort is the point of the question.

Q14. Your NumPy pipeline runs out of memory on an operation that "shouldn't" copy. What are the usual causes?

(1) A broadcast *output* that's huge even though inputs are small — pairwise ops like `X[:, None] - X[None, :]` materialize (n, n, d). (2) Chained expressions allocating temporaries (`(a*b + c)**2` makes several full-size intermediates; `np.multiply(a, b, out=buf)`-style or in-place ops help). (3) Silent float64 upcasts doubling footprint. (4) Fancy indexing copying where a view was assumed. Fixes: chunk the computation, restructure the algebra (norms + matmul instead of explicit differences), control dtypes, use in-place operations.

Pandas

Q15. Explain inner/left/right/outer joins. After a left join, when does the row count of the result exceed the left table's?

Inner keeps keys present in both; left keeps all left rows with NaN for unmatched right columns; right mirrors; outer is the union. A left join grows beyond `len(left)` exactly when the right table has duplicate join keys — each left row multiplies by its number of right matches (Cartesian per key). Guard with `validate='many_to_one'` (raises on unexpected duplicates) and a row-count assertion after the merge.

Q16. groupby: when do you use `transform` instead of `agg`? Give a feature-engineering example.

`agg` returns one row per group — summaries. `transform` returns a same-length result broadcast to every member row — group-relative features without a merge: `df['ratio_to_user_mean'] = df.amount / df.groupby('user_id').amount.transform('mean')`, z-scores within group, group counts as a column (`transform('size')`). If you find yourself computing `agg` and merging it back on the key, `transform` was the answer.

Q17. What is the `SettingWithCopyWarning`, and what's the correct pattern?

It fires on chained indexing assignment — `df[df.x > 0]['y'] = 1` — where the first selection may return a copy, so the write may modify a temporary and vanish. Whether it's a view or copy is an implementation detail you must not rely on. Correct: one indexer, `df.loc[df.x > 0, 'y'] = 1`. When deriving a sub-DataFrame to work on, take `.copy()` explicitly and silence the ambiguity by construction.

Q18. A column of integers gains missing values and becomes float. Why, and what are the options?

Classic pandas uses NumPy's NaN — a float — as its missing marker, and there's no NaN in fixed-width ints, so the column upcasts to float64 (ids like 10234 become 10234.0, and exact equality/joins can break). Options: nullable dtypes (`Int64`, capital I) which carry a separate validity mask; fill the missingness before casting back; or restructure so the key column never has gaps. Knowing *why* (dtype system, not a bug) is the point.

Q19. You must compute a per-row result and your `apply(axis=1)` takes minutes. Walk through the escalation ladder.

(1) Rewrite as vectorized column arithmetic — most row-wise lambdas are just column expressions in disguise (Listing 6: 2,188×). (2) Conditionals → `np.where`/`np.select`; membership → `.isin`; lookups → `.map(dict)`. (3) Strings/dates → `.str`/`.dt` accessors. (4) Drop to NumPy: `f(df.a.values, df.b.values)`. (5) If irreducibly per-row and Python, consider `itertuples()` (much faster than apply's Series construction) or parallelize/chunk. Mention profiling first — optimize the measured bottleneck, not the assumed one.

Q20. How do you process a 100 GB CSV on a 16 GB laptop?

Stream: `pd.read_csv(..., chunksize=1_000_000)` returns an iterator of DataFrames — aggregate incrementally (running sums/counts, groupby partials merged at the end). Cut width early with `usecols`; shrink dtypes (`float32`, `category` for low-cardinality strings — often 10× on its own). Better storage: convert once to Parquet (columnar, compressed, column-pruning reads). Beyond pandas: Polars/DuckDB handle out-of-core elegantly; Spark (Chapter 28) when it's genuinely distributed. The structure of the answer — stream, shrink, change format, change engine — matters more than tool names.

Complexity and performance

Q21. Why is building a string with `+=` in a loop $O(n^2)$, and what's the fix?

Strings are immutable: each `+=` allocates a fresh string and copies both operands, so total copying is $1+2+\ldots+n \approx n^2/2$. Fix: append parts to a list (amortized O(1) each) and `''.join(parts)` once — O(n) total. The same immutability logic explains why repeated `np.append`/`pd.concat` in a loop is quadratic: collect, then concatenate once.

Q22. `x in container` — give the complexity for list, set, and sorted list, and an ML situation where the difference matters.

List: O(n) scan. Set/dict: O(1) average hash lookup. Sorted list with `bisect`: O(log n). Situation: filtering a million tokens against a stopword collection — list makes it O(n·m), set makes it O(n); or checking "have I seen this user id?" during streaming dedup. The follow-up trap: set membership requires hashable elements, so you can't put lists or NumPy arrays in a set (use tuples).

Q23. Why is `deque` the right structure for a sliding window over a stream, and what's the complexity story?

`collections.deque` is a doubly-linked block list: O(1) append and pop at *both* ends, versus a list's O(n) `pop(0)`. A fixed-length window is `deque(maxlen=k)` — appending auto-evicts the oldest element. Streaming feature computations (rolling means over the last k events, rate limiting, recent-history caches) are exactly this access pattern. A list-based window pays O(k) per step; deque pays O(1).

Q24. Estimate memory: a Python list of 10M floats vs a float64 NumPy array vs float32. Why the differences?

NumPy float64: 10M × 8 B = 80 MB. float32: 40 MB. Python list: ~24–28 B per float object (object header + refcount + value) plus 8 B per list pointer → ~300 MB, roughly 4× the float64 array. This is why "load into NumPy with an appropriate dtype" is itself a memory optimization, and why float32 is the deep-learning default (Chapter 13 covers mixed precision).

Clean, testable ML code and scenarios

Q25. Your training metric changed between two runs with identical code and data. List the suspects.

Unseeded (or partially seeded) randomness: NumPy/Python/framework RNGs each need seeding, and a seed set in one process doesn't reach DataLoader workers. Nondeterministic GPU kernels (atomics; cuDNN autotune picking different algorithms). Environment drift: library versions changing defaults. Data order: filesystem listing order or hash-seed-dependent iteration feeding an order-sensitive pipeline. Uncontrolled train/val split. The systematic answer: fix all seeds, pin versions, log configs and data hashes, then diff the two runs' logs. *Interviewers listen for: multiple RNGs and worker processes, not just "set the seed."*

Q26. How would you unit-test a feature preprocessing pipeline? Name test categories with examples.

Contract/shape tests (output is (n, d) float32, column order stable, no NaNs escape); statistical property tests (standardized columns have mean≈0/std≈1; encoded categories inverse-transform back); leakage tests (fit on train, transform a held-out point, verify train statistics were used — Listing 7); edge cases (constant column, empty group, unseen category at transform time, all-missing column); golden tests (tiny fixture in, pinned expected frame out); and an end-to-end smoke test on 100 rows through the whole pipeline. Small deterministic fixtures, one behavior per test.

Q27. What does "overfit one batch" test, and why is it the first debugging step for a new training loop?

Train on ~10 examples until loss approaches zero. A correctly wired model can memorize 10 points; if it can't, the bug is structural — loss connected to the wrong tensors, labels misaligned with inputs (a shuffle applied to X but not y), gradients not flowing (detached graph, wrong optimizer params), learning rate absurd, or wrong loss for the target encoding. It isolates "the code is broken" from "the model/data/hyperparameters are weak" in minutes, before any expensive full run. It's a *necessary* not sufficient check — passing it says nothing about generalization.

Q28. A teammate's 400-line training script works but nobody can modify it safely. What refactor do you propose, concretely?

Extract the stage boundaries first: `load_data() → build_features() → build_model(cfg) → train(model, data, cfg) → evaluate(model, data)`, each pure-ish with explicit inputs/outputs; hyperparameters and paths into one dataclass config (constructible from YAML/CLI); randomness passed in as seeded generators; I/O quarantined at the edges. Then lock behavior with a golden test (fixed seed, tiny data, pinned metric) *before* refactoring, refactor incrementally, and keep the test green. The estimator pattern and the golden-test-first move are what the question screens for.